Национальный исследовательский университет "Высшая школа экономики"
Факультет компьютерных наук
Образовательная программа "Программная инженерия"

СОГЛАСОВАНО	УТВЕРЖДЕНО
Старший преподаватель: Департамент анализа данных и искусственного интеллекта, НИУ ВШЭ	Академический руководитель образовательной программы "Программная инженерия", старший преподаватель департамента программной инженерии
_____ А. А. Паринов	_____ Н. А. Павлочев
«___» _____ 2026 г.	«___» _____ 2026 г.

ЦИФРОВОЙ АССИСТЕНТ СТУДЕНТА: СЕРВЕРНАЯ ЧАСТЬ

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.12.17 81 01-1-ЛУ

Исполнители:
Студент группы БПИ236
_____ / С. Е. Растворов /
«___» _____ 2026 г.

2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

УТВЕРЖДЕН

RU.17701729.12.17 81 01-1-ЛУ

ЦИФРОВОЙ АССИСТЕНТ СТУДЕНТА: СЕРВЕРНАЯ ЧАСТЬ

Пояснительная записка

RU.17701729.12.17 81 01-1

Листов 25

2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

АННОТАЦИЯ

Пояснительная записка содержит сведения о назначении программы, области ее применения, технических характеристиках, выбранных средствах реализации, архитектурных решениях и ожидаемых технико-экономических показателях разработки.

Документ подготовлен в составе программной документации курсового проекта «Цифровой Ассистент Студента: серверная часть» в соответствии с требованиями Единой системы программной документации.

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ . . . 4
1.1. Наименование программы . . . 4
1.2. Документ(ы), на основании которого(ых) ведется разработка . . . 4
2. НАЗНАЧЕНИЕ РАЗРАБОТКИ . . . 5
2.1. Функциональное назначение . . . 5
2.2. Эксплуатационное назначение . . . 5
2.3. Краткая характеристика области применения программы . . . 5
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ . . . 7
3.1. Постановка задачи на разработку программы . . . 7
3.2. Описание и обоснование выбора состава технических и программных средст . . . 7
3.2.1. Состав технических и программных средств . . . 7
3.2.2. Обоснование выбора технических и программных средств . . . 8
3.3. Описание и обоснование архитектуры программы . . . 9
3.4. Особенности функционала . . . 10
3.4.1. Основной сценарий работы . . . 10
3.4.2. Подтверждение email и вход в систему . . . 10
3.4.3. Каталог и карточка проекта . . . 11
3.4.4. Управление проектом заказчиком . . . 11
3.4.5. Подача и обработка заявок . . . 11
3.4.6. Модерация и кабинет ЦППРП . . . 12
3.4.7. Импорт проектов из XLSX . . . 12
3.4.8. Семантический поиск и рекомендации . . . 12
3.4.9. Домен faculty и сопоставление научных руководителей . . . 12
3.4.10. Outbox и синхронизация внешних сервисов . . . 13
3.4.11. Контур развертывания и серверные окружения . . . 13
3.4.12. Непрерывная интеграция и поставка . . . 13
3.4.13. Наблюдаемость, резервное копирование и восстановление . . . 14
3.4.14. Контрольные точки доступа и эксплуатационные ссылки . . . 15
3.5. Обоснование выбора метода организации входных и выходных данных . . . 16
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ . . . 18
4.1. Ориентировочная экономическая эффективность . . . 18
4.2. Предполагаемая потребность . . . 18
4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами . . . 19
ОПИСАНИЕ ПРИМЕНЕНИЯ ГЕНЕРАТИВНОЙ МОДЕЛИ . . . 21
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ . . . 22
ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ . . . 23
ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ . . . 24

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы – «Цифровой Ассистент Студента: серверная часть».

Наименование программы на английском языке – «Digital Student Assistant».

1.2. Документ(ы), на основании которого(ых) ведется разработка

Разработка ведется на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденной академическим руководителем программы темы курсового проекта.

2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

2.1. Функциональное назначение

Программа «Цифровой Ассистент Студента: серверная часть» предназначена для автоматизации работы с проектами НИУ ВШЭ на уровне серверной логики и программных интерфейсов. Система обеспечивает ведение каталога проектов, импорт данных о проектах из XLSX, обработку заявок студентов, поддержку избранного и интересов, выдачу рекомендаций и организацию модерации со стороны сотрудников ЦППРП. Также программа обеспечивает синхронизацию доменных данных между основным веб-приложением, сервисом рекомендаций и графовым сервисом за счет публикации событий изменения состояния.

Функциональное назначение программы включает выполнение следующих задач: хранение и обновление сведений о проектах, ролях и профилях пользователей; предоставление интерфейсов для создания и редактирования карточек проектов; обработку действий студентов по просмотру, поиску, подаче и отзыву заявок; поддержку рабочих операций заказчиков и сотрудников ЦППРП; формирование данных для последующей аналитической и поисковой обработки. В рамках текущей реализации входные данные поступают через веб-интерфейс и API, а выходные данные возвращаются в виде HTML-страниц, JSON-ответов и служебных данных для внешних вспомогательных сервисов.

2.2. Эксплуатационное назначение

Программа предназначена для эксплуатации в учебной среде НИУ ВШЭ студентами, преподавателями и внешними заказчиками проектов, а также сотрудниками ЦППРП, отвечающими за сопровождение проектного процесса. Для студентов система используется как единая точка доступа к каталогу проектов, рекомендациям, личным интересам, избранному и заявкам. Для заказчиков программа служит средством управления карточками проектов, публикацией, квотами и рассмотрением заявок. Для сотрудников ЦППРП она обеспечивает модерацию, контроль сроков, обработку импортированных данных и подготовку выгрузок.

Эксплуатация программы предполагает работу через современный веб-браузер и штатную серверную инфраструктуру на основе Docker Compose, PostgreSQL и Nginx. Доступ к личным функциям ограничивается ролями пользователя; для регистрации и входа используется локальная авторизация по электронной почте с подтверждением адреса, а обработка критичных действий сопровождается журналированием и проверкой прав доступа. Специальная техническая подготовка от пользователя не требуется, однако сотрудникам, работающим с административными разделами, необходимы навыки работы с веб-интерфейсом и выгрузками данных.

2.3. Краткая характеристика области применения программы

Программа применяется в предметной области организации и сопровождения студенческих проектов в образовательной среде. Она ориентирована на процессы подбора, согласования, модерации и распределения проектов между студентами и заказчиками, а также на административное сопровождение проектного фонда и контроль статусов заявок. Область применения охватывает учебные и организационные сценарии, в которых требуется централизованно хранить сведения о

проектах, обновлять их из табличных источников, обрабатывать обращения пользователей и поддерживать актуальное состояние данных.

С точки зрения состава пользователей система рассчитана на три основные группы: студентов, которые выбирают и отслеживают проекты; заказчиков и преподавателей, которые размещают и ведут проекты; сотрудников ЦППРП, которые выполняют модерацию, управление сроками и контроль данных. Дополнительно программа поддерживает домен faculty для синхронизации сведений о преподавателях, публикациях и курсах, а также для сопоставления научных руководителей с проектами. Таким образом, область применения программы определяется не только хранением каталога, но и полным циклом работы с проектами от импорта и публикации до рассмотрения заявок и последующего обновления связанных данных.

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Программа Цифровой Ассистент Студента: серверная часть предназначена для автоматизации работы с учебными проектами НИУ ВШЭ на уровне серверной логики и связанных с ней веб-интерфейсов. В рамках текущей реализации система должна обеспечивать ведение каталога проектов, обработку заявок студентов, управление карточками проектов со стороны заказчиков и сотрудников ЦППРП, импорт проектных данных из табличных файлов, а также синхронизацию данных для сервисов поиска, рекомендаций и графовой проекции.

Входные данные программы формируются из нескольких источников: действий пользователей в веб-интерфейсе, HTTP-запросов к API, файлов импорта формата XLSX, событий доменной модели и служебных сообщений внешним вспомогательным контурам. К программе предъявляются следующие основные ограничения: разграничение доступа по ролям; проверка корректности и полноты данных до выполнения операций; устойчивость к повторной отправке запросов; сохранение целостности проектных, пользовательских и заявочных данных; поддержка развертывания в контейнерной среде.

Ожидаемый результат работы программы состоит в следующем:

- предоставление пользователю актуального каталога проектов и карточек с полным описанием;
- возможность подачи, отзыва, рассмотрения и изменения статусов заявок;
- возможность создания, редактирования, модерации, публикации и снятия проектов;
- импорт и нормализация данных из XLSX-таблиц;
- формирование ответов API в структурированном виде;
- передача доменных изменений в сервисы рекомендаций, поиска и графовой проекции через outbox-механизм;
- ведение журналов и метрик, достаточных для контроля работы системы и диагностики ошибок.

В текущем проектном составе аутентификация реализуется через локальную регистрацию и вход по электронной почте с подтверждением адреса; интеграция с вузовским SSO может рассматриваться как отдельное расширение.

3.2. Описание и обоснование выбора состава технических и программных средст

3.2.1. Состав технических и программных средств

Серверная часть программы разворачивается в контейнерной среде Docker и работает за обратным прокси Nginx. Основной контур включает веб-приложение на Django 5+ и Django REST Framework, отдельный ML-сервис на FastAPI и графовый сервис на FastAPI с хранилищем Neo4j. Для рабочего контура используется PostgreSQL; для изоляции и повторяемости развертывания применяются Docker Compose и конфигурационные файлы окружения.

К составу программных средств относятся:

- Python 3.13+ как основной язык серверной разработки;
- Django 5+ как каркас модульного монолита и шаблонизированной веб-части;

- Django REST Framework для построения API;
- PostgreSQL как основная реляционная база данных;
- Neo4j как хранилище графовой проекции;
- FastAPI для независимых вспомогательных сервисов;
- Nginx как точка входа в систему и прокси;
- Docker и Docker Compose как средства упаковки и запуска;
- средства тестирования, линтинга и типизации, используемые в репозитории;
- Git и GitHub как средства контроля версий и хранения исходных кодов.

Клиентская часть представляет собой веб-интерфейс, доступный из современного браузера. Для просмотра и взаимодействия используются HTML5, CSS3 и JavaScript, а рендеринг основных экранов выполняется шаблонами Django.

3.2.2. Обоснование выбора технических и программных средств

Выбор Django в качестве основы серверной части обусловлен тем, что проект требует не только API, но и полноценный веб-интерфейс с ролями, формами, административными экранами, шаблонами и встроенными механизмами авторизации. Модульный монолит на Django позволяет сохранять единый источник истины для доменных данных, упрощает контроль транзакций, уменьшает количество межсервисных зависимостей и делает сопровождение курсового проекта более предсказуемым.

Django REST Framework выбран для публикации API, поскольку он обеспечивает строгую схему обмена данными, удобную валидацию входных структур и совместимость с существующей архитектурой веб-приложения. Использование Python 3.13+ соответствует требованиям технического задания, обеспечивает доступ к современным языковым возможностям и упрощает поддержку кода в рамках единого репозитория.

PostgreSQL используется как основная СУБД, так как система опирается на транзакционные операции с проектами, заявками, пользователями, интересами, избранным, импортом и модерационными действиями. Для такого набора данных важны надёжность, предсказуемая согласованность, поддержка индексов и возможность логически разделять домены по схемам. Разделение данных по доменным схемам уменьшает связанность и упрощает развитие подсистем без прямых междоменных соединений.

Neo4j применён для графовой проекции, поскольку проект содержит связи, удобные для представления в виде графа: пользовательские и проектные сущности, связи научных руководителей, публикаций и курсов, а также производные отношения, которые могут обновляться независимо от основного транзакционного контура. Такой подход позволяет отделить аналитическую и рекомендательную обработку от основного CRUD-ядра.

FastAPI используется для вспомогательных сервисов, которым требуется легкий HTTP-контур и автономный жизненный цикл. ML-сервис и графовый сервис не должны дублировать бизнес-логику основной серверной части; они получают события и данные по контракту, а затем выполняют специализированную обработку. Это уменьшает риск смешения обязанностей и делает интеграции более прозрачными.

Docker Compose и Nginx выбраны для того, чтобы запускать весь стек воспроизводимо, без зависимости от локальной конфигурации среды разработчика. Контейнеризация также упрощает проверку сборки, резервное развертывание, восстановление и эксплуатационные процедуры, включая синхронизацию конфигураций и резервное копирование базы данных.

3.3. Описание и обоснование архитектуры программы

Архитектура программы построена как модульный монолит Django + DRF с выделенными внешними сервисами для ML и графовой проекции. Основная серверная часть хранит бизнес-логику, доменные модели, правила переходов статусов, формы, административные действия, API-контракты и шаблонные страницы. Внутренние приложения разделены по функциональным доменам: `base`, `users`, `projects`, `applications`, `account`, `imports`, `outbox`, `recs`, `faculty`, `frontend`.

Структурно система включает следующие слои:

- слой представления, который обслуживает HTML-страницы, формы и API-эндпоинты;
- слой прикладной логики, где реализованы переходы статусов, проверки прав, сценарии модерации, обработка заявок и импорт;
- слой доменной модели, содержащий сущности проектов, пользователей, заявок, интересов, избранного, модерации и faculty-домена;
- слой инфраструктуры, отвечающий за работу с PostgreSQL, файловыми импортами, почтой, outbox-доставкой, логированием и внешними сервисами.

Центральная роль отведена приложению `projects`, поскольку именно оно определяет жизненный цикл проектной карточки и изменения, которые далее должны быть отражены в смежных подсистемах. Приложение `applications` отвечает за заявки студентов и связанные с ними переходы статусов. Приложение `users` хранит профиль, роль, интересы, избранное и состояние подтверждения email. Приложение `faculty` обслуживает синхронизацию преподавателей, публикаций, курсов и сопоставление научных руководителей с проектами.

Обмен данными между основной серверной частью и внешними сервисами организован через outbox API доставки событий. Это означает, что изменение в транзакционной части сначала фиксируется в основной базе, после чего как доменное событие попадает в outbox и становится доступным внешним потребителям. Такой способ организации снижает риск рассинхронизации между каталогом проектов, индексом рекомендаций и графовой проекцией, а также позволяет контролировать обработку событий по контрольным точкам и подтверждениям.

Веб-интерфейс строится на шаблонах Django и использует серверный рендеринг для основных экранов. Такой выбор оправдан тем, что в проекте важны предсказуемость отображения, простая интеграция с формами и ролями, а также возможность работать через обычный браузер без отдельного клиентского приложения.

Для административной части используется `django-unfold`, что улучшает работу с модерацией, справочниками и служебными экранами без изменения доменной модели. Разделение на роли `student`, `customer`, `cpprp/staff` позволяет ограничивать видимость маршрутов, форм и API-операций в соответствии с полномочиями пользователя.

3.4. Особенности функционала

Функционал программы организован вокруг нескольких устойчивых пользовательских и серверных сценариев. Каждый сценарий имеет собственные входные данные, набор проверок и ожидаемый результат. Для серверной части критичны валидация, соблюдение прав доступа, транзакционная целостность и формирование доменных событий для смежных сервисов; для клиентской части критичны понятная навигация, корректная обработка ошибок и доступность действий в зависимости от роли.

3.4.1. Основной сценарий работы

Основной сценарий работы системы заключается в последовательном прохождении пользователем полного цикла взаимодействия с проектом: авторизация, просмотр каталога, анализ карточки проекта, подача заявки или редактирование проекта, переход в модерацию и последующее изменение статуса. Для студента этот сценарий заканчивается отправкой заявки и отслеживанием её статуса; для заказчика — созданием и сопровождением проекта; для сотрудника ЦППРП — проверкой карточки, возвратом на доработку или публикацией.

В рамках сценария сервер выполняет проверку прав доступа, формирует ответы в формате JSON либо HTML, записывает изменения в PostgreSQL и, если затронуты доменные сущности, создаёт события для outbox-доставки. Если пользователь работает со страницей каталога, данные фильтруются и пагинируются на сервере. Если выполняется заявка или модерационное действие, система дополнительно проверяет допустимость перехода статуса и фиксирует причину отклонения или возврата, когда она обязательна по правилам процесса.

Результатом сценария является либо новое состояние проектной или заявочной сущности, либо уведомление об ошибке валидации, правах доступа или конфликте статусов. Во всех случаях система должна возвращать детализированную реакцию, достаточную для повторного выполнения действия без потери данных.

3.4.2. Подтверждение email и вход в систему

Раздел подтверждения email используется при регистрации пользователя и при подготовке его к работе в системе. Входными данными являются адрес электронной почты, пароль и код подтверждения, отправленный на email. Система проверяет корректность формата адреса, наличие учётной записи, состояние подтверждения и актуальность кода.

После успешного подтверждения email пользователь получает возможность авторизоваться и обращаться к защищённым API. До подтверждения адреса доступ к выдаче токена ограничивается, что снижает риск использования незавершённых учётных записей и обеспечивает контроль над регистрационным потоком. В административной части и в кабинетах ролей могут отображаться сообщения о необходимости завершить подтверждение.

Выходным результатом сценария является создание или активация учётной записи, изменение статуса подтверждения и подготовка пользователя к дальнейшей работе в системе.

3.4.3. Каталог и карточка проекта

Каталог проектов предназначен для просмотра доступных тем студентами и для контроля списка проектов сотрудниками и заказчиками. Входными данными служат параметры фильтрации, сортировки и поиска: курс, образовательная программа, технологии, тип проекта, статус комплектования, а также текстовый запрос. Сервер применяет фильтры, выполняет пагинацию и возвращает набор карточек с кратким представлением.

Карточка проекта содержит основное описание, контакты, дедлайны, количество заявок, сведения об утверждённых участниках и другие атрибуты, необходимые для принятия решения пользователем. При открытии карточки сервер получает идентификатор проекта, проверяет его доступность и возвращает полное представление, пригодное для отображения в браузере и для дальнейших действий, например подачи заявки или редактирования.

Для student-ролей каталог является основным инструментом выбора, для customer-ролей — инструментом контроля собственных проектов, для staff-ролей — источником модерационных действий и служебной информации.

3.4.4. Управление проектом заказчиком

Заказчик работает с карточкой проекта через сценарий создания, редактирования и отправки на модерацию. Входными данными являются поля формы проекта: название, описание, сроки, теги технологий, квоты мест, контакты и, при необходимости, вложения. Сервер проверяет обязательность полей, корректность форматов и допустимость перехода статуса.

После сохранения черновика заказчик может отправить проект на модерацию. Дальнейшая обработка зависит от состояния карточки: проект может оставаться в черновике, находиться на модерации, быть опубликованным или возвращённым на доработку. История изменений и комментарии модератора сохраняются как часть процесса сопровождения, что позволяет восстановить последовательность действий и причины отказов.

Этот сценарий завершается либо успешным переводом проекта в следующий статус, либо выдачей ошибки валидации или прав доступа.

3.4.5. Подача и обработка заявок

Сценарий заявок связывает студента и проект. Входными данными выступают идентификатор проекта, действие пользователя и, при необходимости, комментарий или подтверждение операции. Студент может подать заявку, отозвать её и отслеживать изменение статуса. Заказчик и сотрудники ЦППРП могут просматривать поступившие заявки и изменять их состояние в пределах своей роли.

При отклонении заявки система должна зафиксировать причину, если это предусмотрено сценарием обработки. При подтверждении заявки состояние заявочной сущности переходит в согласованное значение, а связанные данные становятся доступными для последующих административных процедур. Все изменения должны выполняться с проверкой текущего состояния, чтобы исключить неконсистентные переходы и повторную обработку одной и той же заявки.

3.4.6. Модерация и кабинет ЦППРП

Кабинет сотрудника ЦППРП предназначен для контроля очереди проектов, принятия решений по публикации и сопровождения дедлайнов. Входными данными служат список проектов на модерации, служебные комментарии, параметры сроков и выбор действия модератора. Если проект возвращается на доработку, комментарий должен соответствовать ограничениям, заданным процессом; если проект публикуется, система фиксирует изменение статуса и делает его доступным в каталоге.

Дополнительно кабинет используется для просмотра агрегированных показателей: количество проектов, количество заявок и число утверждённых участников. Такие данные нужны для оперативного контроля и для подготовки административных выгрузок. Интерфейс администратора в проекте опирается на `django-unfold`, что делает работу с очередями и справочниками более удобной без изменения логики обработки данных.

3.4.7. Импорт проектов из XLSX

Импорт используется для загрузки проектных данных из табличных файлов. Входными данными выступают XLSX-файлы и структурированные таблицы, из которых необходимо извлечь проектные карточки, связанные справочники и параметры импортируемых сущностей. При обработке выполняются проверка структуры файла, очистка значений, нормализация строк и контроль обязательных полей.

После успешного импорта система создаёт или обновляет доменные записи и формирует события о завершении импорта. Эти события далее могут быть использованы внешними компонентами для обновления поискового индекса, рекомендаций и графовой проекции. Если файл содержит ошибки, программа должна вернуть детализированное сообщение с указанием проблемных строк или полей.

3.4.8. Семантический поиск и рекомендации

Подсистема поиска и рекомендаций предоставляет фасад над ML-сервисом. На вход подаются текстовый запрос пользователя, параметры области поиска и данные о предпочтениях, сформированные из интересов и избранного. Основная серверная часть обращается к сервису рекомендаций, а при отсутствии семантического результата использует резервное ранжирование по ключевым словам.

Результатом сценария является список релевантных проектов и, при наличии соответствующей логики сервиса, пояснение о причине попадания проекта в выдачу. После изменения проектных данных или выполнения импорта подсистема получает событие на переиндексацию, чтобы актуализировать результат поиска в допустимый срок.

3.4.9. Домен faculty и сопоставление научных руководителей

Домен `faculty` используется для хранения и синхронизации сущностей, связанных с преподавателями, публикациями и курсами. Входными данными являются записи о людях, публикациях, авторствах, курсах и состоянии синхронизации. Система поддерживает обновление этих данных и построение связи между проектом и потенциальным научным руководителем.

Назначение подсистемы состоит в том, чтобы использовать данные факультетского домена для уточнения структуры проекта и автоматизации сопоставления руководителей с профилями НИУ ВШЭ. Изменения в этом домене также должны попадать в outbox, чтобы внешние сервисы могли обновить свои представления без прямого доступа к транзакционной базе.

3.4.10. Outbox и синхронизация внешних сервисов

Подсистема outbox обеспечивает выдачу доменных событий внешним потребителям. Входными данными являются события изменения проектов, заявок, профилей пользователей, дедлайнов, импорта, пересчета рекомендаций и изменений faculty-домена. После фиксации события оно становится доступным через API выдачи, подтверждения и механизмы контрольных точек.

Выходными данными являются подтвержденные события, по которым ML-сервис и графовый сервис актуализируют свои структуры. Такой механизм нужен для отделения транзакционной части от производных представлений и для уменьшения риска потери данных при обновлении вспомогательных сервисов.

3.4.11. Контур развертывания и серверные окружения

В ходе разработки серверной части был сформирован отдельный эксплуатационный контур для локальной работы, staging-стенда и production-стенда. Базовая локальная конфигурация используется для разработки и функциональной отладки; staging-окружение описывается файлом `infra/docker-compose.staging.yml` и включает полный набор сервисов `web`, `ml`, `graph`, `postgres`, `neo4j` и `nginx`; production-окружение описывается файлом `infra/docker-compose.prod.yml` и ориентировано на основной пользовательский контур `web + postgres + nginx` с обязательным выполнением миграций, сбором статических файлов и проверкой готовности сервиса перед приемом внешнего трафика.

Для обоих серверных окружений подготовлены шаблоны переменных окружения `infra/.env.staging.example` и `infra/.env.prod.example`, а также служебный скрипт `infra/sync-env-to-vm.sh` для переноса конфигурации на виртуальную машину. В staging- и production-конфигурациях используется запуск через `gunicorn`, внешний обратный прокси `Nginx`, отдельные тома для `PostgreSQL`, статических файлов и состояния начальной загрузки данных. HTTPS-режим обеспечивается прокси-слоем и доверенными origin-настройками Django; в compose-конфигурациях предусмотрены healthcheck-проверки и ограничения контейнеров (`read_only`, `tmpfs`, `no-new-privileges`, `cap_drop`), что повышает устойчивость и предсказуемость эксплуатации.

3.4.12. Непрерывная интеграция и поставка

Для серверной части настроена цепочка непрерывной интеграции и поставки на основе GitHub Actions. Workflow `CI` запускается при `push` и `pull request`, определяет изменившиеся подсистемы и в зависимости от состава изменений выполняет репозиторный линтинг, проверку `pre-commit`, модульные и интеграционные тесты веб-, ML- и graph-компонентов, а также единый release gate в dockerized-окружении. Дополнительно выполняются security scan и сбор артефактов покрытия для серверной части.

Production-доставка оформлена отдельным workflow `Deploy Production`. При его запуске выполняются сборка и публикация production-образа в `GHCR`, затем по SSH производится обновление серверного рабочего каталога, проверка и дозаполнение критичных переменных окружения, запуск `docker compose` и завершающая проверка уже на целевом сервере.

3.4.13. Наблюдаемость, резервное копирование и восстановление

Для контроля состояния и восстановления работоспособности подготовлены отдельные эксплуатационные средства. Для наблюдаемости используется observability-контур `infra/observability/docker-compose.observability.yml`, который поднимает `Prometheus` и `Grafana`. Метрики формируются серверными сервисами проекта с использованием `prometheus-client`, а конфигурация `Grafana` и готовая панель `dsa-overview` хранятся в репозитории и разворачиваются автоматически через provisioning-механизм.

Для защиты данных подготовлены сценарии резервного копирования и восстановления `scripts/backup-postgres.sh`, `scripts/backup-stack.sh`, `scripts/restore-postgres.sh` и `scripts/install-backup-timer.sh`. При создании резервной копии формируется отдельный каталог запуска, архив дампа `PostgreSQL`, контрольная сумма `sha256` и манифест с параметрами окружения и временем создания. Скрипт восстановления поддерживает работу как с обычным SQL-файлом, так и с архивом `gzip`, системный timer позволяет автоматизировать ночные резервные копии для `prod` и `staging`.

На рисунке приведена панель мониторинга `Grafana`, применяемая для контроля состояния сервисов и readiness-проверок.

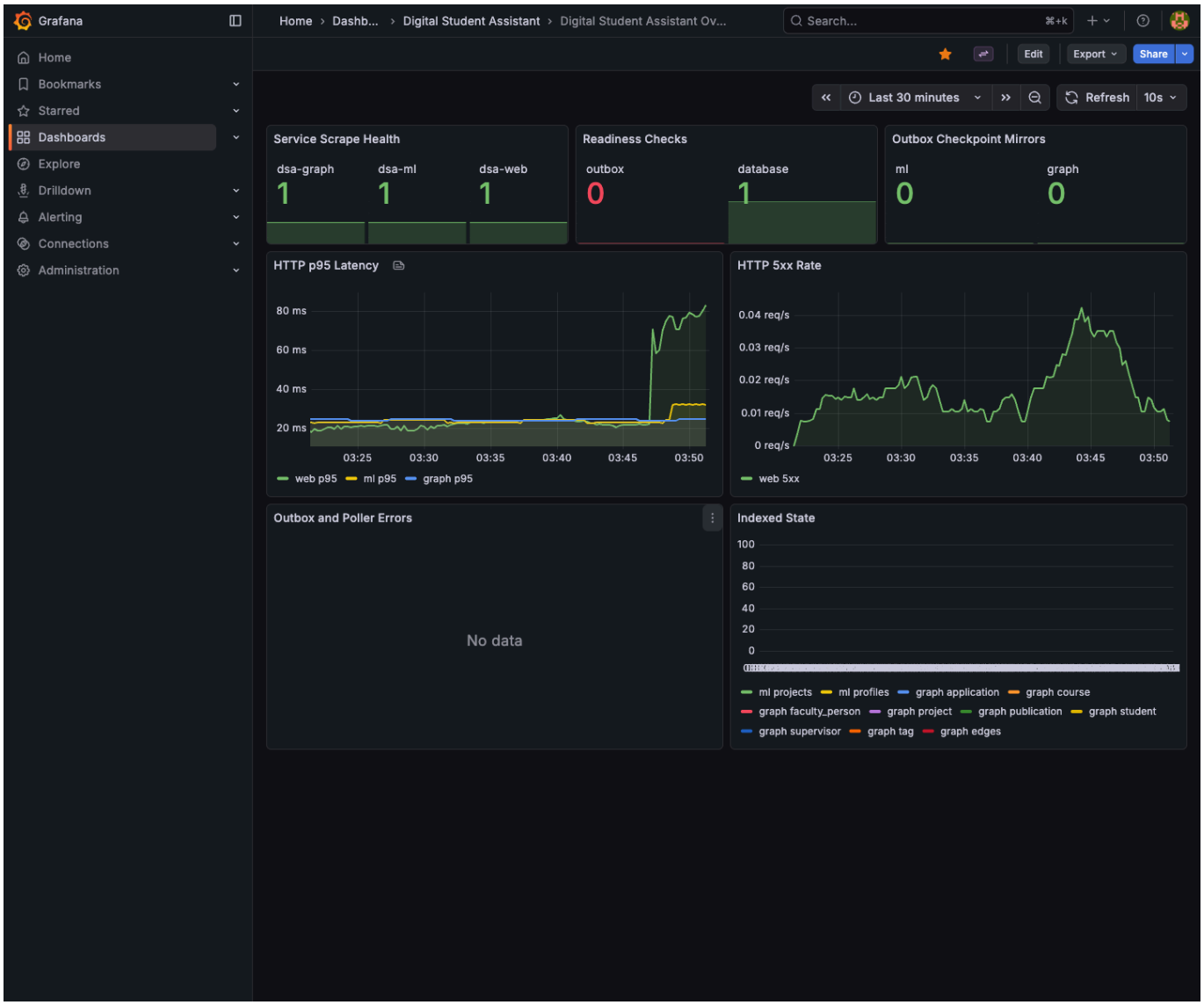

Рис. 1. Панель мониторинга Grafana

3.4.14. Контрольные точки доступа и эксплуатационные ссылки

Для контроля работоспособности системы используются как локальные, так и публичные контрольные точки доступа. Для локального стенда применяются: главная страница `http://127.0.0.1:8000/`, health-проверка `http://127.0.0.1:8000/health/`, корень API `http://127.0.0.1:8000/api/v1/`, документация OpenAPI `http://127.0.0.1:8000/api/docs/`, готовность ML-сервиса `http://127.0.0.1:8001/ready` и состояние graph-сервиса `http://127.0.0.1:8002/state`. Для развернутых внешних стендов используются публичные адреса `https://dsa-staging.s3rg.ru/` для staging-окружения и `https://dsa.s3rg.ru/` для production-окружения; соответственно, контрольные точки API и документации доступны по адресам `https://dsa-staging.s3rg.ru/api/v1/`, `https://dsa-staging.s3rg.ru/api/docs/`, `https://dsa.s3rg.ru/api/v1/` и `https://dsa.s3rg.ru/api/docs/`.

Указанные адреса используются как эксплуатационные ориентиры при приемке, демонстрации и последующей проверке развернутых контуров. При этом шаблоны файлов окружения продолжают содержать обобщенные доменные значения, что не влияет на фактическое описание уже подготовленных стендов в составе пояснительной записки.

На рисунке приведена автоматически сформированная документация OpenAPI, доступная на staging-стенде и используемая для проверки прикладных и служебных маршрутов API.

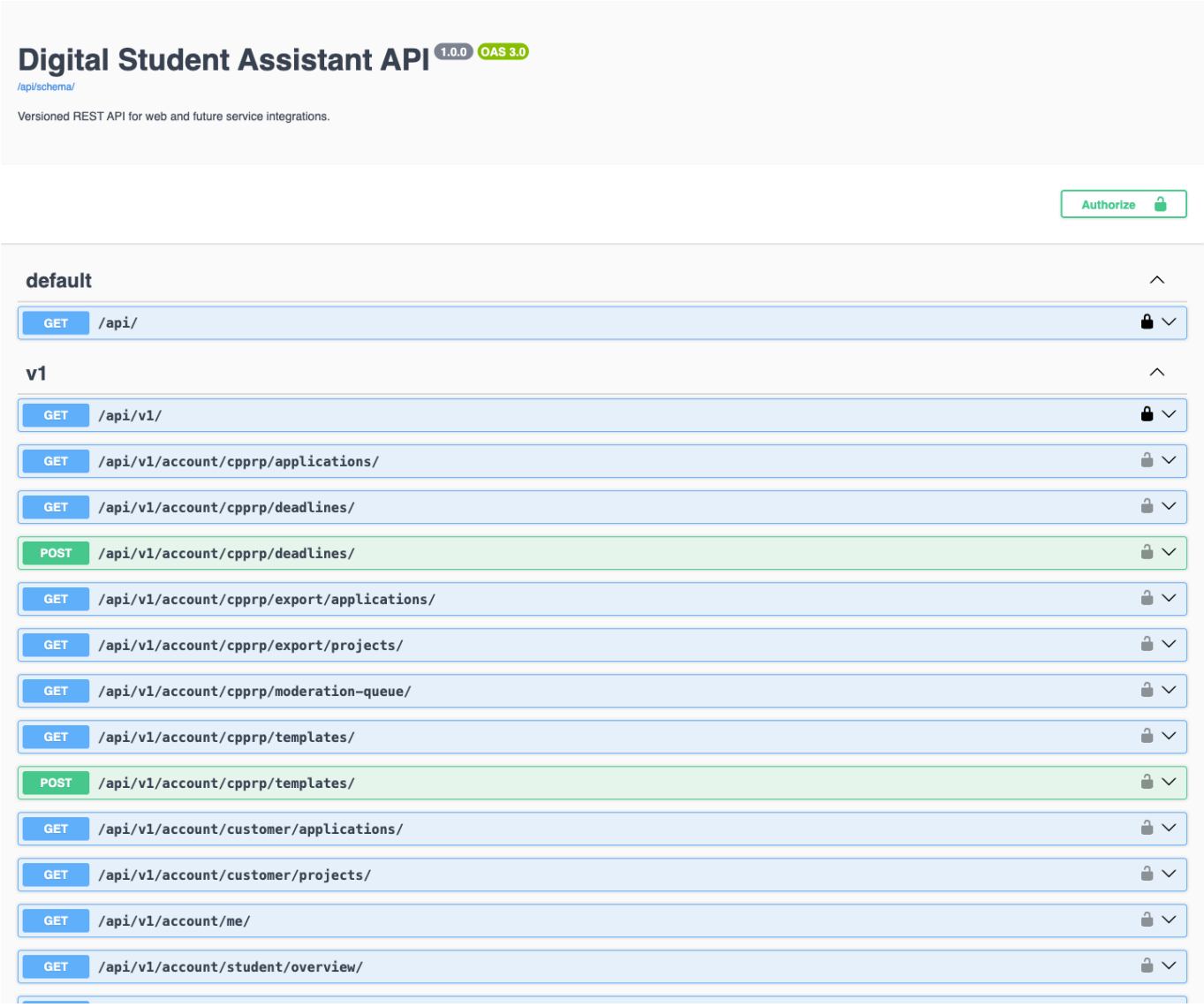

Рис. 2. Документация OpenAPI на staging-стенде

3.5. Обоснование выбора метода организации входных и выходных данных

Входные данные программы организованы по принципу разделения на пользовательские запросы, файловые импорты и служебные события. Пользовательские данные поступают через формы веб-интерфейса и REST/JSON-запросы; они включают учётные данные, параметры фильтрации, поля карточек проектов, комментарии модерации, данные заявок, а также сведения об интересах

и избранном. Файловые данные поступают в виде XLSX-таблиц и проходят этапы чтения, проверки структуры, нормализации и сохранения в доменную модель.

Внутренняя организация данных основана на реляционной модели PostgreSQL с разделением по доменным схемам. Такой подход позволяет хранить связанные сущности компактно и управляемо, не смешивая данные разных подсистем в одной логической области. Между модулями обмен выполняется через прикладные сервисы и внутренние контракты, а не через прямой доступ к таблицам других доменов.

Выходные данные представлены несколькими формами:

- HTML-страницы веб-интерфейса для работы через браузер;
- JSON-ответы API с кодами HTTP и структурированными ошибками;
- служебные ответы подсистемы outbox и внешних сервисов;
- экспортные файлы XLSX или CSV для административной выгрузки;
- журналы и метрики для контроля эксплуатации.

Для обработки ошибок используется явная структура ответа, в которой различаются ошибки валидации, отсутствие прав, отсутствие ресурса, логические конфликты и внутренние ошибки сервера. Это позволяет клиентской части однозначно интерпретировать результат и корректно отображать сообщение пользователю.

В проекте поддерживается асинхронная актуализация производных данных: после изменения основных сущностей формируются события, которые затем используются для обновления индекса рекомендаций и графовой проекции. Такой метод организации входных и выходных данных уменьшает связанность компонентов, делает систему более устойчивой к частичным сбоям и обеспечивает проверяемое обновление всех зависимых представлений.

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Ориентировочная экономическая эффективность

Разработка серверной части системы «Цифровой Ассистент Студента» не ориентирована на прямую коммерческую выручку; ее ожидаемая экономическая эффективность выражается в снижении трудоемкости организационных процессов, связанных с ведением каталога проектов, приемом заявок, модерацией и подготовкой выгрузок для административных процедур.

Наиболее заметный эффект достигается за счет того, что данные вводятся и обрабатываются в едином контуре. Импорт проектов из XLSX, ведение карточек, маршрутизация заявок, фиксация статусов и работа с ролями выполняются в одной системе, а не через набор разрозненных таблиц и ручных согласований. Это уменьшает число повторных операций, сокращает время на перенос сведений и снижает вероятность ошибок, связанных с дублированием или несогласованностью данных.

Дополнительный экономический эффект дают:

- сокращение времени на обработку пакета проектов и заявок в пиковые периоды;
- уменьшение затрат на исправление ошибок, вызванных ручным вводом;
- снижение нагрузки на сотрудников, сопровождающих проектный цикл;
- ускорение поиска подходящего проекта студентами за счет фильтрации, семантического поиска и рекомендаций;
- уменьшение затрат на эксплуатацию за счет контейнеризованного развертывания, резервного копирования и скриптов обслуживания.

Для учебного подразделения эффект проявляется прежде всего как сокращение операционной трудоемкости и повышение прозрачности процесса. Для пользователей системы эффект выражается в более быстром доступе к актуальным данным и в уменьшении числа действий, необходимых для подачи заявки, модерации и контроля статусов.

4.2. Предполагаемая потребность

Потребность в системе определяется регулярным проведением проектного набора и необходимостью сопровождения большого объема карточек проектов и заявок. По материалам проекта, в пиковый период нагрузка соответствует примерно 1700 студентам, что требует устойчивой работы каталога, форм заявок, механизмов поиска и административных операций.

Разработка востребована следующими группами пользователей:

- студентами, которым требуется единый каталог проектов, рекомендации, избранное и отслеживание статусов заявок;
- преподавателями и внешними заказчиками, которые создают и сопровождают проектные карточки;
- сотрудниками ЦППРП, которые выполняют модерацию, контролируют дедлайны, обрабатывают заявки и формируют выгрузки;
- научными руководителями, для которых важно сопоставление проектов с профилями HSE, публикациями и курсами.

Практическая необходимость в системе обусловлена тем, что проектный процесс включает несколько ролей и большое число переходов между статусами. При отсутствии единого инструмента

эти операции обычно выполняются вручную, через таблицы, формы и переписку, что увеличивает риск расхождений данных и делает процесс хуже масштабируемым. Разрабатываемое решение устраняет указанные недостатки за счет централизованного хранения данных, ролевой модели доступа и формализованных сценариев обработки.

4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами

Экономические преимущества решения определяются тем, что система объединяет полный цикл сопровождения студенческих проектов: импорт данных, каталог, модерацию, заявки, избранное, интересы, рекомендации, административные выгрузки и домен faculty для связи проектов с научными руководителями. Это уменьшает объем ручной координации между отдельными инструментами и снижает стоимость сопровождения процесса в течение всего учебного цикла.

По сравнению с текущим процессом, основанным на Excel-таблицах и веб-формах, проект исключает повторный ввод сведений, уменьшает количество ошибок и обеспечивает прозрачный контроль статусов. По сравнению с существующими институциональными решениями проект ориентирован именно на единый пользовательский сценарий для студентов, заказчиков и ЦППРП, а не на разрозненное выполнение отдельных административных операций.

Функция	Текущий процесс ФКН через Excel-таблицы и веб-формы	Цифровой Ассистент Студента: серверная часть
Единый каталог проектов и просмотр карточек	-	+
Импорт проектов из XLSX и структурированных таблиц	-	+
Роли студентов, заказчиков и ЦППРП	-	+
Подача и обработка заявок	-	+
Избранное, интересы и рекомендации	-	+
Модерация, сроки и статусы публикации	-	+
Административные выгрузки и сводные показатели	-	+
Итого	**0**	**7**

Таблица 1. Сравнение функциональных характеристик с ближайшими аналогами

ОПИСАНИЕ ПРИМЕНЕНИЯ ГЕНЕРАТИВНОЙ МОДЕЛИ

При подготовке данной курсовой работы генеративные модели использовались как вспомогательный инструмент при работе с текстовой документацией. Использование не носило автономного характера: итоговые формулировки, отбор фактов, проверка соответствия техническому заданию, сопоставление с исходным кодом и окончательная редактура выполнялись авторами работы.

Генеративная модель применялась для следующих целей:

- подготовки черновых формулировок отдельных фрагментов пояснительной записки, программы и методики испытаний, руководства оператора и связанной программной документации;
- унификации терминологии и стиля изложения в разных документах комплекта;
- проверки согласованности между текстом пояснительной записки, техническим заданием и фактическим составом функций проекта;
- редактирования русскоязычных технических формулировок и сокращения числа стилистических несоответствий;
- подготовки черновых вариантов отдельных фрагментов программного кода, конфигурации и служебных сценариев при последующей ручной доработке.

В работе использовалась генеративная модель OpenAI Codex, доступная через инструментальную среду Codex. Адрес источника в сети «Интернет»: https://openai.com/. Способ применения модели: формирование черновых вариантов текста, предложение альтернативных формулировок, редактура отдельных разделов, выявление потенциальных противоречий между документами, а также подготовка черновых фрагментов кода и конфигурации. Использование модели не означало автоматического принятия сгенерированного результата: все предложенные фрагменты программного кода, конфигурации и документации проходили ручную проверку, адаптацию к структуре проекта и последующее тестирование либо сверку с исходным кодом. Содержательные решения о составе системы, описании архитектуры, перечне функций и результатах испытаний принимались авторами работы на основании исходного кода, технического задания и сопутствующей документации проекта.

С точки зрения состава работы использование генеративной модели относится не ко всей работе целиком, а к отдельным фрагментам текста, программного кода и конфигурации. Наибольший объем применения пришелся на разделы, связанные с выравниванием стиля, согласованием терминологии, подготовкой служебной документации и получением черновых вариантов технической реализации. Оценка эффективности использования модели является положительной: применение модели позволило ускорить подготовку черновых формулировок, техническую редактуру и предварительную кодогенерацию, однако не заменяло ручную проверку фактов, сверку с кодовой базой, контроль соответствия ТЗ и окончательное авторское редактирование.

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. Правила использования искусственного интеллекта студентами НИУ ВШЭ. URL: https://www.hse.ru/studyspravka/ai_guidelines/ (дата обращения: 29.04.2026).
15. Положение об организации промежуточной аттестации и текущего контроля успеваемости студентов Национального исследовательского университета «Высшая школа экономики» (ПОПАТКУС). URL: https://www.hse.ru/studyspravka/ai_guidelines/ (дата обращения: 29.04.2026).

ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ

Приложение	Ссылка
ИПС ФЭН	https://fes-projects.ru/
Ярмарка элементов практической подготовки	https://archive-smartpro.hse.ru/
Текущий процесс ФКН через веб-формы	https://forms.yandex.ru/

Дата обращения: 29.04.2026.

ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ

Термин	**Определение**
Каталог проектов	Раздел системы, в котором пользователю доступны список проектов, фильтрация, поиск и переход к карточкам проектов.
Карточка проекта	Структурированное описание проекта, включающее название, содержание, контакты, сроки, количество мест и иные служебные атрибуты.
Заявка	Запись о намерении студента участвовать в выбранном проекте, содержащая текущий статус и историю обработки.
Модерация	Проверка проектной карточки сотрудником ЦППРП с последующей публикацией, отклонением или возвратом на доработку.
Outbox	Подсистема фиксации и выдачи доменных событий для последующей обработки внешними сервисами.
ML-сервис	Отдельный сервис, обеспечивающий семантический поиск и рекомендации по проектам.
Graph-сервис	Отдельный сервис, формирующий графовую проекцию связей между сущностями проекта.
Faculty-домен	Подсистема хранения сведений о преподавателях, публикациях, курсах и сопоставлении научных руководителей с проектами.
REST/JSON API	Прикладной интерфейс системы, использующий HTTP и формат JSON для обмена данными между компонентами и внешними потребителями.
Ролевая модель	Механизм разграничения доступа, определяющий допустимые действия пользователя в зависимости от его роли.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					